Scalable Computing: Practice and Experience
Volume 1, Issue 1, 2026
SCPE | *Journal Article* **OPEN ACCESS**

S^3: Spectral-Spatial-Smooth Fine-Tuning for Large Language Models on 4 GB Consumer GPUs

Nicolás Andrey Orozco Giraldo[1,*], **Luis Eduardo Muñoz Guerrero**[1] **and Yony Fernando Ceballos**[2]

[1] *Universidad Tecnológica de Pereira, Pereira, Colombia*
[2] *Facultad de Ingeniería, Grupo de Ingeniería y Sociedad, Universidad de Antioquia, Medellín, Colombia*
[*] *Corresponding author: nicolas.orozco@utp.edu.co*

How to cite:
Author. Title. *Scalable Comput. Pract. Exp.* **2026**, *1*(1).

DOI:
10.1234/scpe.2026.001

Check for updates

Article History:
Received: –
Accepted: –
Published: –

Abstract: Fine-tuning large language models with parameter-efficient methods such as low-rank adaptation typically requires eight to sixteen gigabytes of GPU memory, since even a low-rank adapter trains against a frozen base model resident in memory at full size. We introduce S3 (Spectral-Spatial-Smooth), a method that adapts seven-billion-parameter models on four-gigabyte consumer GPUs by combining disk-streamed execution with three coupled operators. A Singular Value Modulation Operator modulates each singular value of a frozen weight matrix through a learned nonlinear function, leaving the singular vectors untouched, with a parameter count independent of model width. A Neural Manifold Flow deforms latent representations through a continuous-depth neural ordinary differential equation. A Spectral Transport Bridge couples the two by routing representations through the same spectral basis the first operator modulates, creating shared information between them that would otherwise be zero. Together they total 3.90 million trainable parameters, about a twentieth of one percent of the base model. On a four-gigabyte GPU, peak memory ranges from 1.6 to 3.4 gigabytes depending on workload. Across seven zero-shot tasks, it reaches accuracy comparable to a standard low-rank baseline while training several times fewer parameters. An ablation shows the spectral pathway alone matches the flow pathway with fewer parameters at matched budget, and combining both beats either alone. Re-evaluating a trained model with one to sixteen integration steps instead of four changes held-out perplexity by only a few thousandths of a percent, evidence the learned flow behaves as a continuous system rather than a shortcut tied to one step count.

Keywords: parameter-efficient fine-tuning; large language models; spectral adaptation; neural ordinary differential equations; disk-streamed inference; consumer GPU

1. Introduction

1.1 Motivation: Who Actually Needs This

Fine-tuning large language models for downstream tasks is the dominant paradigm in applied NLP [1, 2]. A 7–8B parameter model needs over 60 GB of GPU memory to fine-tune outright, which puts it firmly in data-center territory. LoRA [1], QLoRA [2], DoRA [3], and VeRA [4] bring that down substantially, but the numbers typically reported (8–16 GB) still sit above what a card like the GTX 1050 (4 GB) offers.

Google Colab's free tier gives out a 16 GB GPU, plenty for LoRA on a 7B model with the usual tooling. Anyone with steady access to that has little reason to reach for S^3 instead; LoRA on a free cloud GPU is simpler and probably faster than what we report here. Who actually needs this: people without the connectivity to hold a cloud session open for hours, anyone whose training data cannot leave the device, users in places where those cloud services are restricted, jobs that must run unattended longer than a free tier allows, and anyone running this often enough that an owned GPU beats paying for a subscription every month. That is a real group of people, just a smaller one than "anyone without much money."

For that audience we push the VRAM floor down further than LoRA or QLoRA do, aiming for hardware the intended user is likely to already own, on the order of $100–150 used, with no rented accelerator required.

The 4 GB budget targeted here is not itself the ceiling. The disk-streaming substrate S^3 is built on bounds VRAM by the size of the single largest unit processed at a time: a per-layer bound, independent of total model depth or parameter count. S^3 inherits this bound instead of re-deriving it, since the three operators added on top are a small, fixed overhead that does not grow the way the frozen base model's own footprint would. Our own project notes work out what this implies for a 70B-parameter model on the same card (on the order of 200 MB of streamed factors per layer, 30–36 hours for three epochs); these are design estimates made before this paper's experiments, and everything actually measured here is on Qwen2.5-7B. We report the estimate because it is the reason 4 GB matters beyond this one model: 7B is only the first checkpoint of a larger claim.

1.2 Our Approach: Three Mutually Coupled Operators

Departing from the dominant paradigm of additive low-rank perturbations in the original weight space [1], S^3 operates in three distinct domains: the spectral domain of weight matrices, the manifold of latent representations, and a bridge that couples the two. Three operators, derived from first principles, cover each domain:

1. **SVMO (Singular Value Modulation Operator).** Given a frozen weight matrix $W = U\Sigma V^T$, SVMO learns a pointwise nonlinear function $m_\theta : \mathbb{R}_+ \to \mathbb{R}$ that modulates each singular value σ_i independently, producing $W_{\text{adapted}} = U \cdot m_\theta(\Sigma) \cdot V^T$. U and V remain permanently frozen; only the scalar singular values are modulated. The modulation function is a compact 2-hidden-layer MLP with only $O(H^2)$ parameters per matrix, independent of model dimension d and SVD rank k. This differs from Zhang & Pilanci's Spectral Adapter [7], which applies additive shifts or orthogonal rotations to singular vectors with $O(dk)$ parameters.

2. **NMF (Neural Manifold Flow).** Existing residual adapters apply a static perturbation, $h' = h + \text{MLP}(h)$, a single-step translation. NMF instead defines a continuous deformation via a Neural ODE [5], $dh(t)/dt = f_\theta(h(t), t)$, solved with fixed-step RK4 integration, letting

representations follow curved trajectories through the latent manifold that a discrete single-step adapter cannot produce.

3. **STB (Spectral Transport Bridge).** SVMO modulates weight spectra; NMF deforms latent representations. Without a coupling mechanism, these adaptations proceed independently and cannot coordinate. STB bridges them via cross-attention in the spectral basis: it projects the latent representation into the left singular vector basis ($h \mapsto U_k^T h = s$), cross-attends that spectral signature against the current modulated singular values, and maps back to latent space via U_k. This bidirectional channel lets weight-space and representation-space adaptations coordinate during training.

Together, these three operators constitute a mutually reinforcing adaptation stack. Table 1 works out a generic 32-layer, d=4096 architecture to **4.83M** parameters (0.060% of 7B) as a design illustration; on the model we actually train and evaluate throughout Section 4 (Qwen2.5-7B-Instruct, 28 layers, d=3584), the measured trainable count is **3.90M** (**0.0511%**). Measured peak VRAM on a real GTX 1050 (4 GB card) ranges **1.6–3.4 GB** depending on workload (Table 4); the ∼345 MB figure in Table 2 is a per-layer analytical decomposition, not an end-to-end measurement, and Section 3.4.3 discusses the gap between the two directly.

1.3 Theoretical Contributions

We establish ten formal results that underpin the S^3 design:

- **Theorem 1** (Approximation Rate): Explicit $O(1/\sqrt{H})$ error bound for SVMO via a universal approximation lemma for 2-hidden-layer GELU networks, with concrete numerical evaluation showing ∼4% relative error at $H = 32$.
- **Theorem 2** (Gradient Stability): Uniform gradient norm bound via tanh saturation, guaranteeing training stability absent from LoRA-style methods.
- **Theorem 3** (Corrected Grönwall): Linear-in-T trajectory deviation for NMF ODE flows (correcting the commonly misapplied exponential form), with explicit Lipschitz constant $L_f \approx 5 \times 10^{-4}$.
- **Theorem 4** (RK4 Accuracy): Global integration error $O(\Delta t^4)$ with $\varepsilon_{\text{ODE}} \leq 6.3 \times 10^{-6}$ for $N = 4$ steps, well below fp16 precision.
- **Theorem 5** (Mutual Information): Proof that STB creates non-zero shared information $I \geq (\beta^2 k)/(2d) \cdot H(X)$ between SVMO and NMF adaptations; without STB, $I = 0$ via the data processing inequality.
- **Theorem 6** (Convergence): Local Hessian conditioning improvement via spectral preconditioning through STB with explicit bound on $\kappa_{\text{STB}}/\kappa_{\text{no-STB}}$.
- **Theorem 7** (PAC-Bayes): Non-vacuous generalization bound with effective dimension $d_{\text{eff}} \approx$ 1,760 and $\text{KL}(Q\|P_c) \leq$ 7,320, giving a 26.5% generalization gap: the first such guarantee for PEFT at 7B scale.
- **Theorem 8** (SGC): Exact gradient preservation under spectral compression, so SVMO gradients are **identical** when computed from the full gradient or from the compressed gradient $g \cdot U_k U_k^T$ (32× compression, zero information loss).

- **Theorem 9** (NFR): Linear-in-epoch error accumulation for progressive ODE integration; recycling intermediate ODE states as warm-start initial conditions introduces no exponential error growth.
- **Theorem 10** (SBS): Stochastic STB bypass with probability p_{STB} preserves non-zero mutual information $I_{\text{SBS}} \geq p_{\text{STB}} \cdot \beta^2 k/(2d) \cdot H(X) > 0$ for any $p_{\text{STB}} > 0$.

2. Related Work

2.1 Parameter-Efficient Fine-Tuning

LoRA [1] learns additive low-rank matrices $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$, requiring $O(dr)$ trainable parameters per weight matrix. QLoRA [2] augments LoRA with 4-bit NormalFloat quantization of the frozen base model, reducing VRAM to ∼4–5 GB for 7B-scale fine-tuning. That is closer to our target, but still above the 4 GB GTX 1050 used throughout this paper. DoRA [3] decomposes pretrained weights into magnitude and direction components before applying LoRA-style adaptation. VeRA [4] uses shared frozen random matrices plus two learnable scaling vectors per layer, achieving minimal parameter counts but with limited expressivity. IA^3 [16] learns per-dimension rescaling vectors.

Key distinction. All prior methods apply adaptations exclusively in the *original weight space* or the *latent representation space*, and do so independently. None operates in the spectral domain of pretrained weight matrices, employs continuous-depth ODEs, or couples weight-space and representation-space adaptations through a learned information channel.

2.2 Spectral Methods

Zhang & Pilanci [7] propose the Spectral Adapter, which computes the SVD of pretrained weights and applies either additive shifts to singular values or orthogonal rotations of the top-k singular vectors.

Our SVMO differs fundamentally. (i) U and V are *completely frozen*: we never rotate or shift singular vectors. (ii) The modulation is a *learned, pointwise, nonlinear* function $m_\theta(\sigma_i)$ applied independently to each singular value, in place of an additive constant or uniform rotation. (iii) Our parameter count is $O(H^2)$ per matrix, independent of both model dimension d and SVD rank k, versus $O(dk)$ for the Spectral Adapter. (iv) We provide formal approximation rates (Theorem 1) and gradient stability guarantees (Theorem 2), neither of which appears in prior spectral PEFT work.

2.3 Neural ODEs in Deep Learning

Neural ODEs [5] introduced continuous-depth models where the forward pass solves an initial value problem. Applications include time-series modeling [20], continuous normalizing flows [19], and physical simulation. **Neural ODEs have not previously been applied to parameter-efficient LLM fine-tuning.** Our NMF adapts this framework under a critical design constraint: the velocity field f_θ must be a compact bottleneck MLP ($d_b \in \{4, 8, 16\}$ hidden units) so that ODE integration adds negligible overhead (∼2% of total FLOPs). We further provide the first formal Grönwall-based trajectory analysis and RK4 integration guarantees for ODE-based PEFT (Theorems 3, 4).

2.4 Latent-Space and Coupled Adaptations

Residual adapters that modify hidden states via $h' = h + \mathrm{MLP}(h)$ have been extensively studied [21, 22]. These methods apply *single-step static perturbations*. In contrast, NMF substitutes the static MLP with a continuous ODE flow, enabling curved trajectories and explicitly time-dependent dynamics: a structural increase in expressivity at equivalent parameter counts.

Our STB introduces a spectral cross-attention channel bridging weight-space and representation-space adaptations, a mechanism without direct precedent in the residual-adapter literature we are aware of.

3. Method: The S^3 Architecture

We present the three operators with full formal definitions and complete proofs for all ten theorems.

3.1 Singular Value Modulation Operator (SVMO)

Definition 3.1 (SVMO Forward Pass)**.** *Let $W \in \mathbb{R}^{d_{out} \times d_{in}}$ be a frozen pretrained weight matrix. Compute the compact SVD offline once:*

$$W = U\Sigma V^T, \tag{1}$$

and store only the rank-k factors $U_k \in \mathbb{R}^{d_{out} \times k}$, $\Sigma_k = \mathrm{diag}(\sigma_1, \ldots, \sigma_k)$, $V_k^T \in \mathbb{R}^{k \times d_{in}}$, computed via randomized SVD [6] (approximately 50× faster than full SVD; ∼2 minutes on CPU for a 7B model). For input $x \in \mathbb{R}^{B \times d_{in}}$:

$$y = U_k \cdot \left(m_\theta(\Sigma_k) \odot (xV_k^T)\right) \tag{2}$$

where $\odot$ is broadcast element-wise multiplication along the k-dimension, and $m_\theta(\Sigma_k)$ yields a k-dimensional vector of modulated singular values. The effective adaptation matrix is:

$$\Delta_{SVMO} = W_{SVMO} - W = U_k\big(m_\theta(\Sigma_k) - \Sigma_k\big)V_k^T. \tag{3}$$

Modulation Function Design. The core of SVMO is the learned pointwise modulation:

$$m_\theta(\sigma) = \sigma \cdot \big(1 + \alpha \cdot \tanh\big(g_\theta(\log(\sigma + \varepsilon))\big)\big) \tag{4}$$

where $\alpha \in (0, 1]$ controls the maximum fractional modulation, $\varepsilon = 10^{-8}$, and $g_\theta : \mathbb{R} \to \mathbb{R}$ is a 2-hidden-layer MLP:

$$g_\theta(z) = W_3 \cdot \mathrm{GELU}\left(W_2 \cdot \mathrm{GELU}(W_1 z + b_1) + b_2\right) + b_3 \tag{5}$$

with hidden width $H \in \{16, 32, 64\}$. Total trainable parameters per weight matrix: $K \approx H^2 + 3H + 1 \approx H^2$, **independent of** d **and** k.

Design properties. (1) *Bounded modulation:* $m_\theta(\sigma) \in [\sigma(1-\alpha), \sigma(1+\alpha)]$ via tanh saturation, so singular values cannot be driven to zero or flipped in sign. (2) *Identity initialization:* g_θ initialized with near-zero output ($W_3, b_3 \sim \mathcal{N}(0, 10^{-5})$) so $m_\theta(\sigma) \approx \sigma$ at training start, meaning the adapted model starts from the exact pretrained model. (3) *Log-domain:* operating on $\log \sigma$ compresses the dynamic range of singular values (which can span orders of magnitude) into a numerically well-conditioned domain.

3.1.1 Lemma 1: Approximation Rate for GELU Networks

Lemma 3.2 (Approximation Rate for 2-Hidden-Layer GELU MLPs)**.** *Let $\mathcal{F}_H$ be the family of 2-hidden-layer GELU MLPs from $\mathbb{R} \to \mathbb{R}$ with hidden width H. For any Lipschitz function $f : [a, b] \to \mathbb{R}$ with Lipschitz constant L_f and $\sup_{x\in[a,b]} |f(x)| \leq M_f$, there exists $g_\theta \in \mathcal{F}_H$ such that:*

$$\sup_{x\in[a,b]} |g_\theta(x) - f(x)| \leq \frac{C \cdot L_f \cdot (b-a)}{\sqrt{H}} \tag{6}$$

where $C \leq 12$ is a universal constant, derived from Yarotsky [8] Theorem 1, adapted from ReLU to GELU via the smooth approximation $\mathrm{GELU}(x) \approx x\Phi(x)$ which inherits the same approximation rate up to a constant factor [12].

Sketch. Yarotsky [8] establishes that ReLU networks with 2 hidden layers of width H approximate Lipschitz functions on compact domains with error $O(1/\sqrt{H})$. The constant $C \leq 12$ derives from the refined bound in Theorem 1 therein. Since $\mathrm{GELU}(x) = x\Phi(x)$ where Φ is the Gaussian CDF, a smooth, non-polynomial activation, the universal approximation property for non-polynomial activations [12] guarantees the same asymptotic rate. The smoothness of GELU (Lipschitz, bounded derivative) ensures that the worst-case approximation constant remains within a factor of 2 of the ReLU bound; empirical evaluation across $f \in \{\sin(cx), e^{-x^2}, \tanh^{-1}(x/\alpha)\}$ on $[a, b]$ yields $C \approx 8.4$ for GELU, conservatively bounded by 12. □

3.1.2 Theorem 1: Approximation Capacity of SVMO

Theorem 3.3 (SVMO Approximation Capacity with Explicit Rate)**.** *Let $W \in \mathbb{R}^{d\times d}$ with compact SVD $W = U\Sigma V^T$, $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$, $r = \mathrm{rank}(W)$. Let $W_k = U_k\Sigma_k V_k^T$ be the rank-k truncation. Let $\Delta^* \in \mathbb{R}^{d\times d}$ be any target adaptation matrix with $\|\Delta^*\|_F \leq \delta$. Denote by $\Delta^*_{(>k)}$ the projection of Δ^* onto $\mathrm{span}\{u_i v_j^T\}_{\max(i,j)>k}$ (all entries structurally unreachable by SVMO). Then there exists a parameter choice θ such that:*

$$\|\Delta_{SVMO} - \Delta^*\|_F \leq \frac{C\alpha L_{f^*} \,\mathrm{diam}(K)\, \|\Sigma_k\|_F}{\sqrt{H}} + \|\Delta^*_{(>k)}\|_F \tag{7}$$

where $\mathrm{diam}(K) = |\log(\sigma_1 + \varepsilon) - \log(\sigma_k + \varepsilon)|$, L_{f^*} is the Lipschitz constant of the target modulation function $f^*(z) = \tanh^{-1}(d^*_{ii}/(\alpha\sigma_i))$ on the compact log-singular-value domain, and $C \leq 12$ is the constant from Lemma 3.2.

Concrete prediction. *For H=32, d=4096, k=128, α=0.3: $\mathrm{diam}(K) \approx 3.4$, $\|\Sigma_k\|_F \approx 56$. With $L_{f^*} \approx 3.3$, the first term ≈ 121. Relative to $\|W\|_F \approx 3000$, the **relative error** $\approx 4\%$, well within single-precision noise. Doubling H to 64 halves the bound.*

Proof. The proof proceeds in four steps.

Step 1 — SVD decomposition of Δ^*. Expand Δ^* in the SVD basis of W:

$$\Delta^* = \sum_{i=1}^{r}\sum_{j=1}^{r} d^*_{ij}\, u_i v_j^T \tag{8}$$

where $d^*_{ij} = (U^T\Delta^* V)_{ij}$. Decompose $\Delta^* = \Delta^*_{(\leq k,\,\mathrm{diag})} + \Delta^*_{(>k)}$, where $\Delta^*_{(\leq k,\,\mathrm{diag})} = \sum_{i=1}^{k} d^*_{ii} u_i v_i^T$ (diagonal entries within the top-k block) and $\Delta^*_{(>k)}$ contains *all* entries unreachable by SVMO:

off-diagonal entries d_{ij}^* for $i \neq j$ with $\max(i,j) \leq k$, plus all entries involving indices beyond k. By orthogonality of the SVD basis, $\|\Delta^*\|_F^2 = \|\Delta^*_{(\leq k,\,\mathrm{diag})}\|_F^2 + \|\Delta^*_{(>k)}\|_F^2$.

Step 2 — Error decomposition. $\Delta_{\mathrm{SVMO}} = U_k(m_\theta(\Sigma_k) - \Sigma_k)V_k^T = \sum_{i=1}^k (m_\theta(\sigma_i) - \sigma_i)u_i v_i^T$. Hence Δ_{SVMO} modifies only the first k diagonal entries. The Frobenius error decomposes:

$$\|\Delta_{\mathrm{SVMO}} - \Delta^*\|_F^2 = \underbrace{\sum_{i=1}^{k} (m_\theta(\sigma_i) - \sigma_i - d_{ii}^*)^2}_{\text{diagonal mismatch}} + \|\Delta^*_{(>k)}\|_F^2. \tag{9}$$

Step 3 — Diagonal approximation via Lemma 3.2. For each $i \in \{1,\ldots,k\}$, the required modulation is $m_\theta(\sigma_i) - \sigma_i = d_{ii}^*$, which is equivalent to:

$$g_\theta(\log(\sigma_i + \varepsilon)) = \tanh^{-1}\left(\frac{d_{ii}^*}{\alpha\sigma_i}\right) \equiv f^*(\log(\sigma_i + \varepsilon)). \tag{10}$$

Define the compact interval $K = [\log(\sigma_k + \varepsilon), \log(\sigma_1 + \varepsilon)]$. The target function $f^* : K \to \mathbb{R}$ is Lipschitz. Its Lipschitz constant derives from the derivative of $\tanh^{-1}(x/\alpha)$:

$$\frac{d}{dz} f^*(\log \sigma(z)) = \frac{1}{\alpha} \cdot \frac{1}{1 - (d(z)/(\alpha\sigma(z)))^2}, \tag{11}$$

bounded by $1/(\alpha(1 - (\delta/(\alpha\sigma_k))^2))$. For $\alpha = 0.3$ and $\delta/\sigma_k \ll 1$, $L_{f^*} = O(1/\alpha)$.

By Lemma 3.2, there exists g_θ with:

$$\sup_{z \in K} |g_\theta(z) - f^*(z)| \leq \frac{C \cdot L_{f^*} \cdot \mathrm{diam}(K)}{\sqrt{H}}. \tag{12}$$

Applying tanh (Lipschitz constant 1):

$$\begin{aligned} m_\theta(\sigma_i) - \sigma_i - d_{ii}^* &= \sigma_i\alpha \cdot |\tanh(g_\theta) - \tanh(f^*)| \\ &\leq \sigma_i\alpha \cdot |g_\theta - f^*| \\ &\leq \sigma_i\alpha C L_{f^*}\, \mathrm{diam}(K)/\sqrt{H}. \end{aligned} \tag{13}$$

Step 4 — Assembly. Summing the squared per-singular-value errors across $i = 1,\ldots,k$ and taking the square root:

$$\sqrt{\sum_{i=1}^{k} (\sigma_i\alpha \cdot \varepsilon_g)^2} = \alpha \cdot \varepsilon_g \cdot \|\Sigma_k\|_F, \tag{14}$$

where $\varepsilon_g = C \cdot L_{f^*} \cdot \mathrm{diam}(K)/\sqrt{H}$. By the triangle inequality:

$$\|\Delta_{\mathrm{SVMO}} - \Delta^*\|_F \leq \frac{C\alpha L_{f^*}\, \mathrm{diam}(K)\|\Sigma_k\|_F}{\sqrt{H}} + \|\Delta^*_{(>k)}\|_F. \tag{15}$$

□ □

Practical implications. Increasing H from 32 to 128 reduces the approximation error by $\sqrt{128/32} = 2\times$. The truncation error $\|\Delta^*_{(>k)}\|_F$ decreases as k grows. For $k = 128$ on a $d = 4096$ matrix, the normalized truncation error is typically $< 0.05\|\Delta^*\|_F$ because singular spectra of pretrained LLM weights decay rapidly.

3.1.3 Theorem 2: Uniform Gradient Bound

Theorem 3.4 (Uniform Gradient Stability for SVMO)**.** *Let $\mathcal{L}$ be a differentiable loss. For any input $x \in \mathbb{R}^{B\times d_{in}}$, the gradient norm with respect to SVMO parameters θ satisfies:*

$$\left\|\frac{\partial \mathcal{L}}{\partial \theta}\right\|_2 \leq \alpha \cdot \sigma_{\max} \cdot k \cdot \|g'_\theta\|_\infty \cdot \left\|\frac{\partial \mathcal{L}}{\partial y}\right\|_F \cdot \sqrt{|\theta|} \tag{16}$$

*where $\sigma_{\max} = \max_i \sigma_i$, $\|g'_\theta\|_\infty = \sup_z |g'_\theta(z)|$ (bounded since all MLP weights are finite), and $|\theta| \approx H^2$ is the parameter count of g_θ. The bound is **independent of input scale** $\|x\|_F$ due to* $\operatorname{sech}^2(z) \leq 1$ *saturation of* tanh.

Proof. From the SVMO forward pass $y_{b,i} = \sum_{j=1}^{k} U_{i,j} \cdot m_\theta(\sigma_j) \cdot (xV_k)_{b,j}$, the gradient for parameter θ_p via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_p} = \sum_{j=1}^{k} \frac{\partial m_\theta(\sigma_j)}{\partial \theta_p} \cdot s_j, \tag{17}$$

Bound 1 — Modulation derivative.

$$\frac{\partial m_\theta(\sigma_j)}{\partial \theta_p} = \sigma_j \alpha \cdot \operatorname{sech}^2(g_\theta(\log(\sigma_j+\varepsilon))) \cdot \frac{\partial g_\theta}{\partial \theta_p}. \tag{18}$$

Since $\operatorname{sech}^2(z) = 1 - \tanh^2(z) \leq 1$ for all $z \in \mathbb{R}$, and $\sigma_j \leq \sigma_{\max}$:

$$\left|\frac{\partial m_\theta(\sigma_j)}{\partial \theta_p}\right| \leq \alpha \cdot \sigma_{\max} \cdot \|g'_\theta\|_\infty. \tag{19}$$

Bound 2 — Sensitivity aggregation s_j**.** By Cauchy-Schwarz and orthonormality of U_k ($\|U_k\|_F^2 = k$, entries bounded by 1):

$$|s_j| \leq \left\|\tfrac{\partial \mathcal{L}}{\partial y}\right\|_F \cdot \|xV_k\|_F \leq \left\|\tfrac{\partial \mathcal{L}}{\partial y}\right\|_F \cdot \|x\|_F. \tag{20}$$

Summing over k singular values: $\sum_{j=1}^{k} |s_j| \leq k \cdot \|\partial \mathcal{L}/\partial y\|_F \cdot \|x\|_F$. The input factor $\|x\|_F$ is absorbed into $\|\partial \mathcal{L}/\partial y\|_F$ since $\partial \mathcal{L}/\partial x$ propagates proportionally through the frozen model.

Assembly. For each parameter θ_p:

$$\left|\frac{\partial \mathcal{L}}{\partial \theta_p}\right| \leq \alpha \sigma_{\max} \|g'_\theta\|_\infty \sum_{j=1}^{k} |s_j| \leq \alpha \sigma_{\max} k \|g'_\theta\|_\infty \left\|\frac{\partial \mathcal{L}}{\partial y}\right\|_F. \tag{21}$$

Aggregating over all $|\theta| \approx H^2$ parameters via the ℓ_2-norm:

$$\left\|\frac{\partial \mathcal{L}}{\partial \theta}\right\|_2 \leq \alpha \sigma_{\max} k \|g'_\theta\|_\infty \left\|\frac{\partial \mathcal{L}}{\partial y}\right\|_F \sqrt{|\theta|}. \tag{22}$$

□ □

Concrete values (Llama-3, d**=4096,** k**=128,** H**=32).** $\alpha = 0.3$, $\sigma_{\max} \approx 15$, $\|g'_\theta\|_\infty \leq 10$, k=128, $\sqrt{|\theta|} \approx 32$. Gradient bound factor $\approx 1.8 \times 10^5$. With $\|\partial \mathcal{L}/\partial y\|_F \sim 0.1$–$1.0$, $\|\partial \mathcal{L}/\partial \theta\|_2 \sim 10^4$–$10^5$, well within AdamW's effective range.

Comparison with LoRA. LoRA gradients scale with $O(d \cdot r)$ through the low-rank matrices A, B. SVMO gradients are controlled solely by α, $\sigma_{\max}$, and H, all fixed constants after initialization, which gives **inherent training stability** absent from LoRA-style methods.

3.2 Neural Manifold Flow (NMF)

Definition 3.5 (NMF Continuous Deformation). *Let $h \in \mathbb{R}^d$ be a latent representation at any point in the transformer (post-attention or post-MLP). We define a continuous flow over "virtual time" $t \in [0, T]$:*

$$\frac{dh(t)}{dt} = f_\theta\big(h(t), t\big), \qquad h(0) = h, \qquad \tilde{h} = h(T) \tag{23}$$

where $T \in [0.5, 2.0]$ is the total flow duration. The adapted output is $\tilde{h} = h(T)$. The total deformation induced by NMF:

$$\Phi_\theta(h) = h(T) - h(0) = \int_0^T f_\theta(h(t), t)\, dt. \tag{24}$$

This formulation provides: (1) Continuous trajectories: representations follow curved paths in latent space instead of single-step translations; (2) Time-dependent dynamics: the velocity field can change over t, enabling multi-phase deformations; (3) Smoothness: the flow is at least C^1 if f_θ is differentiable.

Velocity Field Design. f_θ is a bottleneck MLP with **explicit time dependence**:

$$f_\theta(h, t) = W_{\text{out}} \cdot \tanh\big(W_{\text{in}} \cdot [h; t]\big) \tag{25}$$

where $[h; t] \in \mathbb{R}^{d+1}$ is the concatenation of h and scalar t, $W_{\text{in}} \in \mathbb{R}^{d \times d_b}$, $W_{\text{out}} \in \mathbb{R}^{d_b \times d}$, and $d_b \in \{4, 8, 16\}$ is the bottleneck dimension. Total parameters: $2d \cdot d_b$ ($\approx$65K for $d = 4096$, $d_b = 8$). Concatenating t makes the velocity field explicitly time-dependent. Initialization: $W_{\text{out}} \approx 0$ so $f_\theta \approx 0$ initially $\Rightarrow \tilde{h} \approx h$ at training start.

ODE Solver. We use fixed-step Runge-Kutta 4 (RK4) with $N \in \{2, 4, 8\}$ steps and step size $\Delta t = T/N$:

$$\begin{aligned} k_1 &= f_\theta(h_n, t_n) \\ k_2 &= f_\theta(h_n + \tfrac{\Delta t}{2} k_1, t_n + \tfrac{\Delta t}{2}) \\ k_3 &= f_\theta(h_n + \tfrac{\Delta t}{2} k_2, t_n + \tfrac{\Delta t}{2}) \\ k_4 &= f_\theta(h_n + \Delta t \cdot k_3, t_n + \Delta t) \\ h_{n+1} &= h_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned} \tag{26}$$

Each step costs 4 evaluations of the tiny f_θ ($\sim$65K params). With $N = 4$, total overhead: $\approx 2\%$ of a single forward pass FLOPs.

Placement. We apply NMF at two points per transformer layer: $\mathbf{NMF}_1$ after the attention residual (post-W_O, before LayerNorm/RMSNorm) and $\mathbf{NMF}_2$ after the MLP residual (post-W_{down}, before layer output). Total: 32 layers $\times$ 2 = 64 NMF flows.

3.2.1 Lemma 2: Lipschitz Constant for Bottleneck tanh MLP

Lemma 3.6 (Lipschitz Constant of Bottleneck Velocity Field). *For $f_\theta(h, t) = W_{out} \cdot \tanh(W_{in} \cdot [h; t])$ with $W_{in} \in \mathbb{R}^{d \times d_b}$, $W_{out} \in \mathbb{R}^{d_b \times d}$, the Lipschitz constant with respect to h satisfies:*

$$L_f \leq \|W_{out}\|_2 \cdot \|W_{in}\|_2. \tag{27}$$

*η **Lipschitz bound.** For Xavier init $\|W\|_2 \approx \sqrt{2/(d_{in}+d_{out})}$, d=4096, d_b=8:*

$$\|W_{in}\|_2 \approx \sqrt{2/(4096{+}8)} \approx 0.022, \ \|W_{out}\|_2 \approx 0.022. \tag{28}$$

Hence $L_f \leq 0.022 \times 0.022 \approx 4.9 \times 10^{-4}$.

Proof. For any $h_1, h_2 \in \mathbb{R}^d$, the difference in the concatenated inputs is only in the first d components since t is shared. Hence:

$$\begin{aligned} \|f_\theta(h_1,t) - f_\theta(h_2,t)\| &= \|W_o(\tanh u_1 - \tanh u_2)\| && (29)\\ &\le \|W_o\|_2 \|\tanh u_1 - \tanh u_2\| && (30)\\ &\le \|W_o\|_2 \|u_1 - u_2\| \quad (\|\tanh'\|_\infty \le 1) && (31)\\ &= \|W_o\|_2 \|W_i(h_1 - h_2)\| && (32)\\ &\le \|W_o\|_2 \, \|W_i\|_2 \, \|h_1 - h_2\|. && (33) \end{aligned}$$

Here $u_j = W_i[h_j; t]$. Thus $L_f \le \|W_{\text{out}}\|_2 \cdot \|W_{\text{in}}\|_2$. □ □

3.2.2 Theorem 3: NMF Trajectory Bound with Corrected Grönwall

Theorem 3.7 (NMF Approximation Capacity with Corrected Grönwall)**.** *Let $h, h^* \in \mathbb{R}^d$ be original and optimal latent representations. Assume h^* is reachable from h via a C^1 path $\gamma : [0,T] \to \mathbb{R}^d$ with $\gamma(0) = h$, $\gamma(T) = h^*$, and $\|\gamma'(t)\| \le M$. Let f_θ have Lipschitz constant L_f with respect to h. Let $h_{NMF}(T)$ be the RK4-approximated solution. Then there exists θ such that:*

$$\|h_{NMF}(T) - h^*\|_2 \le \varepsilon_1 \cdot \frac{e^{L_f T} - 1}{L_f} + \frac{C_{RK4} \cdot T^5}{N^4} \tag{34}$$

where $\varepsilon_1 = \sup_{t\in[0,T]} \|f_\theta(\gamma(t),t) - \gamma'(t)\|$ is the velocity field approximation error.

Critical correction. *For small $L_f T \ll 1$ (which holds: $L_f \approx 4.9 \times 10^{-4}$, $T = 1.0$), Taylor expansion $\frac{e^x - 1}{x} = 1 + x/2 + x^2/6 + \ldots$ gives:*

$$\|h_{NMF}(T) - h^*\| \approx \varepsilon_1 \cdot T \quad \textit{(linear in } T\textit{, not exponential)} \tag{35}$$

This corrects the commonly misapplied bound $Te^{L_f T}$ [5] which would incorrectly predict exponentially growing error. For our parameters, $e^{L_f T} - 1 \approx 4.9 \times 10^{-4}$ and the factor $(e^{L_f T} - 1)/L_f \approx 1.00025 \approx 1$.

Proof. The proof proceeds in four parts.

Part 1 — Target velocity field. Define $v^*(t) = \gamma'(t)$, the ideal velocity field. The ideal ODE $dh/dt = v^*(t)$ with $h(0) = h$ has solution $h_{\text{ideal}}(t) = \gamma(t)$, so $h_{\text{ideal}}(T) = h^*$. By the fundamental theorem of calculus:

$$h^* = h + \int_0^T v^*(t)\, dt. \tag{36}$$

Part 2 — Velocity field approximation. We must approximate the d-dimensional time-dependent curve $v^* : [0,T] \to \mathbb{R}^d$ using a bottleneck MLP with hidden width d_b. By the localized universal approximation theorem for approximating curves parameterized by $t \in [0,T]$, there exists θ such that:

$$\sup_{t\in[0,T]} \|f_\theta(\gamma(t),t) - v^*(t)\| \le \varepsilon_1, \tag{37}$$

for ε_1 arbitrarily small, provided $d_b \ge C_1 \cdot (1+T)$ where C_1 depends on the smoothness of v^*. For smooth v^* (Lipschitz, bounded) and $T \in [0.5, 2.0]$, C_1 is modest. With $d_b \in \{8, 16\}$, this condition is easily satisfied.

Part 3 — Trajectory deviation via integral Grönwall. Define the error function $e(t) = \|h_{\text{NMF}}(t) - h_{\text{ideal}}(t)\|_2$. Differentiating and applying the triangle inequality:

$$\begin{aligned} e'(t) &\le \|f_\theta(h_{\text{NMF}}(t), t) - v^*(t)\|_2 \\ &\le \underbrace{\|f_\theta(h_{\text{NMF}}(t), t) - f_\theta(h_{\text{ideal}}(t), t)\|_2}_{\le L_f \cdot e(t)} \\ &\quad + \underbrace{\|f_\theta(h_{\text{ideal}}(t), t) - v^*(t)\|_2}_{\le \varepsilon_1} \\ &\le L_f \cdot e(t) + \varepsilon_1. \end{aligned} \tag{38}$$

This is the differential inequality $e'(t) \le L_f e(t) + \varepsilon_1$ with initial condition $e(0) = 0$ (since both start at h). By the **integral form** of Grönwall's inequality [13]:

$$e(t) \le \int_0^t e^{L_f(t-s)} \cdot \varepsilon_1 \, ds = \varepsilon_1 \cdot \frac{e^{L_f t} - 1}{L_f}. \tag{39}$$

Not the pointwise form $te^{L_f t}$ which is commonly but incorrectly applied. Evaluating at $t = T$:

$$e(T) \le \varepsilon_1 \cdot \frac{e^{L_f T} - 1}{L_f}. \tag{40}$$

For our bottleneck MLP (Lemma 3.6), $L_f \approx 4.9 \times 10^{-4}$, $T = 1.0$, so $L_f T \approx 4.9 \times 10^{-4} \ll 1$. Taylor expansion of the Grönwall factor yields $\frac{e^x - 1}{x} \approx 1 + x/2 \approx 1.00025$. Therefore $e(T) \approx \varepsilon_1$: the trajectory error is simply the velocity field approximation error, with **no exponential amplification**.

Part 4 — RK4 integration error. From Theorem 3.8 (proved below), the discrete RK4 solution introduces additional global error $\varepsilon_{\text{ODE}} \le C_{\text{RK4}} \cdot T^5 / N^4$.

Final assembly. Via the triangle inequality:

$$\|h_{\text{NMF}}(T) - h^*\|_2 \le e(T) + \varepsilon_{\text{ODE}} \tag{41}$$

$$\le \varepsilon_1 \cdot \frac{e^{L_f T} - 1}{L_f} + \frac{C_{\text{RK4}} T^5}{N^4}. \tag{42}$$

□ □

Practical significance. The bound proves that NMF error amplification is **linear in** T, not exponential. This validates using flow durations $T \in \{0.5, 1.0, 2.0\}$ without risk of divergence: longer flows only proportionally increase the approximation burden on f_θ instead of exponentially amplifying integration errors.

3.2.3 Theorem 4: RK4 Numerical Stability

Theorem 3.8 (Global Error Bound for RK4-NMF Integration)**.** *Let f_θ be the NMF velocity field with Lipschitz constant L_f with respect to h, using* tanh *activation (all derivatives uniformly bounded). Let $N \ge 2$ be the number of RK4 steps. The global error between the discrete RK4 solution and the true ODE solution satisfies:*

$$\|h_{RK4}(T) - h_{exact}(T)\|_\infty \le \frac{T^5}{N^4} \cdot \frac{M_4}{5} \cdot (e^{L_f T} - 1) \tag{43}$$

where $M_4 = \max_{t \in [0,T]} \|f_\theta^{(4)}(h(t), t)\|_\infty$, bounded because tanh *derivatives decay factorially:* $|\tanh^{(n)}(x)| \le 2^n n!$.

Proof. Standard RK4 convergence analysis [14, 13]. The local truncation error for RK4 at step n is:

$$\tau_{n+1} = \frac{\Delta t^5}{5!} \cdot f_\theta^{(5)}(\xi_n) + O(\Delta t^6) \tag{44}$$

for some intermediate point ξ_n. Since tanh derivatives satisfy $|\tanh^{(5)}(x)| \leq 2^5 \cdot 5! = 3840$, and the bottleneck MLP amplifies by at most $\|W_{\text{out}}\|_2 \|W_{\text{in}}\|_2^5 \approx 0.022^6 \approx 1.1 \times 10^{-10}$, the 5th derivative is dominated by tanh: $M_4 \approx \max_t \|f_\theta^{(4)}\|_\infty \leq 16$ (4th derivative bound).

The global error accumulates with exponential factor $e^{L_f T}$ due to Lipschitz propagation across N steps:

$$\begin{aligned} \|e_N\| &\leq \frac{\Delta t^5}{5} \cdot M_4 \cdot \sum_{i=0}^{N-1} e^{L_f (N-i)\Delta t} \\ &\leq \frac{\Delta t^5}{5} \cdot M_4 \cdot \frac{e^{L_f T} - 1}{e^{L_f \Delta t} - 1} \\ &\leq \frac{\Delta t^4}{5} \cdot M_4 \cdot (e^{L_f T} - 1). \end{aligned} \tag{45}$$

Substituting $\Delta t = T/N$ yields the stated bound. □ □

Corollary 3.9 (Concrete Integration Error)**.** *With $N = 4$, $T = 1.0$, $L_f \approx 4.9 \times 10^{-4}$:*

$$\begin{aligned} \varepsilon_{ODE} &\leq \frac{1}{4^4} \cdot \frac{16}{5} \cdot (e^{4.9 \times 10^{-4}} - 1) \\ &\approx \frac{16}{1280} \cdot 4.9 \times 10^{-4} \\ &\approx 6.3 \times 10^{-6}. \end{aligned} \tag{46}$$

***Integration error is negligible**, well below fp16 machine epsilon ($\sim 10^{-4}$) and fp32 precision ($\sim 10^{-7}$). Even with $N = 2$, $\varepsilon_{ODE} \leq 6.3 \times 10^{-6} \times 16 \approx 1.0 \times 10^{-4}$, still negligible.*

Geometric Intuition. Standard residual adapters apply a static translation $h \to h + \text{offset}$, a straight-line displacement. NMF applies a continuous flow with non-zero curvature: the velocity field can produce large, coarse displacements early in the flow ($t \approx 0$) and fine-grained adjustments later ($t \approx T$). Discrete MLPs cannot generate trajectories with curvature; ODE flows naturally produce curves, spirals, and arbitrary smooth paths. That qualitative expressivity compensates for the small parameter count.

3.3 Spectral Transport Bridge (STB)

3.3.1 The Core Problem: Coupling Disjoint Adaptation Spaces

SVMO operates in the **spectral domain** of weight matrices: it modulates singular values Σ via $U\Sigma V^T$. NMF operates in the **latent representation space**: it flows vectors h through $dh/dt = f_\theta(h, t)$. These are fundamentally disjoint: SVMO sees the internal structure of weights, while NMF sees the external behavior of representations. Without a bridge, they adapt independently: SVMO modulates weights blind to how representations are flowing, and NMF deforms representations unaware of the spectral structure of the weights that process them.

STB creates a **bidirectional coupling** that allows these two adaptation mechanisms to coordinate, sharing information in both directions during training.

3.3.2 Formal Definition

Definition 3.10 (STB Bidirectional Coupling)**.** *Given latent representation $h \in \mathbb{R}^d$ and the left singular vectors $U_k \in \mathbb{R}^{d\times k}$ from SVMO:*

$$\mathrm{STB}_\phi(h) = U_k \cdot c_\phi\big(U_k^T \cdot h,\ \Sigma_k\big) \tag{47}$$

*where $s = U_k^T h$ is the **spectral signature** of h: how h projects onto each principal spectral direction of W. The coupling function $c_\phi : \mathbb{R}^k \times \mathbb{R}^k \to \mathbb{R}^k$ receives both the spectral signature s and the current singular values Σ_k, seeing both worlds simultaneously.*

Coupling Function Design. c_ϕ is a lightweight single-head cross-attention:

$$c_\phi(s, \sigma_{\log}) = s + \beta \cdot W_o\big(\,\mathrm{softmax}\big(\frac{QK^T}{\sqrt{k_h}}\big) \odot V\big) \tag{48}$$

with $Q = W_q s \in \mathbb{R}^{k_h}$ (query from spectral signature), $K = W_k \sigma_{\log} \in \mathbb{R}^{k_h}$ (key from current modulated singular values), $V = W_v s \in \mathbb{R}^{k_h}$ (value from spectral signature), projections $W_q, W_k, W_v \in \mathbb{R}^{k\times k_h}$, $W_o \in \mathbb{R}^{k_h \times k}$, $k_h = \lfloor k/4 \rfloor$, and coupling strength $\beta \in [0, 1]$. Parameters per STB instance: $|\phi| = 3k \cdot k_h + k_h \cdot k \approx 3k^2/4$. For $k = 128$: 12,288 $\approx$ 12K.

The cross-attention **attends the spectral signature against the current modulated singular values**. This creates a *data-dependent spectral transformation*: representations that excite different spectral components get transformed differently.

Dual Role. (1) *Feedback channel (backward):* $\partial\mathcal{L}/\partial\Sigma_k$ flows through STB, linking NMF-induced representation changes to SVMO's weight modulation updates and creating an information loop: NMF deforms $h \to$ STB transports to spectral $\to \Sigma_k \to$ SVMO modulates weights $\to$ next forward pass. (2) *Feedforward preconditioning (forward):* Input representations are spectrally aligned before entering attention/MLP blocks, improving conditioning.

3.3.3 Theorem 5: Mutual Information via STB Coupling

Theorem 3.11 (STB Creates Non-Zero Mutual Information)**.** *Let X be the training data distribution. Let $\Theta_S = \theta_{SVMO}$ and $\Theta_N = \theta_{NMF}$ be the adapter parameters after training, viewed as random variables induced by the training trajectory over X. Then:*

$$I_{\text{noSTB}}(\Theta_S; \Theta_N \mid X) = 0, \tag{49}$$

$$I_{\text{STB}}(\Theta_S; \Theta_N \mid X) \geq \frac{\beta^2 k}{2d} H(X). \tag{50}$$

where $H(X)$ is the entropy of the training data. Without STB, SVMO and NMF are information-theoretically independent; with STB, they share non-zero mutual information through the spectral coupling bottleneck of dimension k.

Proof. **Part (i) — No STB.** Without STB, the Markov structure decomposes as:

$$X \to h \to \begin{cases} \text{SVMO-process} \to \Theta_S \\ \text{NMF-process} \to \Theta_N \end{cases} \tag{51}$$

with two parallel branches and no cross-edges between Θ_S and Θ_N. By the data processing inequality [11], conditioning on the intermediate representation h (a deterministic function of X through the frozen base model) breaks any dependence:

$$I(\Theta_S; \Theta_N \mid X, h) = 0. \tag{52}$$

Since h is a deterministic function of X,

$$I(\Theta_S; \Theta_N \mid X) \leq I(\Theta_S; \Theta_N \mid X, h) = 0, \tag{53}$$

and mutual information is non-negative by definition. Hence $I_{\text{no-STB}} = 0$.

Part (ii) — With STB. The Markov chain includes the coupling step:

$$X \to h \xrightarrow{U_k^T} s \xrightarrow{c_\phi(\cdot, \Sigma_k)} \tilde{s} \xrightarrow{U_k} h_{\text{STB}}. \tag{54}$$

The coupling function creates a channel of bandwidth k where Σ_k (controlled by SVMO) and s (derived from NMF-deformed representations) interact.

The gradient of the loss with respect to Σ_k flows through STB backpropagation. This Jacobian is non-zero when $\beta > 0$.

By the information bottleneck principle [18], the STB channel of dimension k can transmit at most k bits of shared information per batch, with effective capacity modulated by coupling strength β. Treating the cross-attention as a Gaussian channel of dimension $k_h = k/4$ with SNR $\propto \beta^2$:

$$I_{\text{STB}} \geq \frac{k_h}{2} \log(1 + \beta^2) \approx \frac{\beta^2 k}{2d} \cdot H(X), \tag{55}$$

where the factor $1/d$ accounts for the dimensional reduction from the d-dimensional latent space to the k-dimensional spectral bottleneck. □ □

Concrete values. With $k = 128$, $\beta = 0.5$, $d = 4096$: $I_{\text{STB}} \geq (0.25 \times 128)/(2 \times 4096) \cdot H(X) \approx 0.0039 \cdot H(X)$. Even this small fraction of shared information accumulates across thousands of gradient steps (τ iterations multiply effective mutual information accumulation), enabling coordinated adaptation.

Significance. Theorem 3.11 proves that STB creates **non-zero coordination** between SVMO and NMF: without STB, the two adaptation mechanisms cannot coordinate, since Theorem 3.11 guarantees $I = 0$. The coupling enables mutually reinforcing fine-tuning: weight modulation and representation flow discover complementary strategies that reinforce each other.

> **Remark (scope of Theorems 5–6).** The bounds above are qualitative ("> 0"), not quantitative guarantees of a large effect. With our default hyperparameters (k=128, β=0.5, d=4096), the mutual-information lower bound evaluates to $I_{\text{STB}} \gtrsim 0.0039 \cdot H(X)$ and the Hessian-conditioning improvement to $\kappa_{\text{STB}}/\kappa_{\text{no-STB}} \approx 0.992$: both real, both numerically small. The theorems establish that STB *can* couple the two operators; they do not establish that this coupling is the dominant driver of quality. What supports that stronger claim is empirical: Section 4 shows the spectral pathway (SVMO+STB) needs STB to move perplexity at all (SVMO alone: +0.02 ppl for 226K extra params; SVMO+STB: +0.25 ppl for 685K).

3.3.4 Theorem 6: Local Convergence Acceleration

Theorem 3.12 (Local Convergence Acceleration via Spectral Preconditioning)**.** *Let $\mathcal{L}(\Theta)$ be the training loss, where $\Theta = (\theta_S, \theta_N)$ are SVMO and NMF parameters. Near a local minimum Θ^*, the Hessian $H = \nabla^2 \mathcal{L}(\Theta^*)$ governs the local convergence rate of gradient descent.*

Without STB, the block Hessian is:

$$H_{\textit{no-STB}} = \begin{bmatrix} H_{SS} & O(1/B) \\ O(1/B) & H_{NN} \end{bmatrix} \tag{56}$$

where off-diagonal terms $O(1/B)$ vanish as batch size grows (consequence of Theorem 3.11, part i). The condition number $\kappa(H_{\textit{no-STB}}) = \max\{\kappa(H_{SS}), \kappa(H_{NN})\}$. For pretrained LLM weights, $\kappa(H_{SS})$ is bounded below by the largest-to-k-th singular value ratio: $\kappa(H_{SS}) \geq \sigma_1/\sigma_k$.

With STB, the coupling channel introduces non-zero cross-terms of strength β:

$$H_{STB} = \begin{bmatrix} H_{SS} & \beta \cdot H_{SN} \\ \beta \cdot H_{NS} & H_{NN} \end{bmatrix} \tag{57}$$

where H_{SN} captures how changes in NMF parameters affect SVMO gradients (and vice versa) through the STB backpropagation path. The cross-terms improve Hessian conditioning via the Schur complement mechanism. Then:

$$\kappa_{STB} \leq \min\left\{1 + \frac{\beta^2 k}{d},\ \frac{1}{\beta^2}\right\} \cdot \kappa_{no\text{-}STB} \tag{58}$$

For $\beta = 0.5$, $k = 128$, $d = 4096$: the min *term is dominated by $1 + 0.25 \times 128/4096 \approx 1.0078$, yielding $\kappa_{STB}/\kappa_{no\text{-}STB} \approx 0.992$, a modest but theoretically guaranteed improvement.*

Sketch. The cross-term H_{SN} arises from differentiating the STB forward chain: $\partial^2\mathcal{L}/\partial\theta_S\partial\theta_N = \partial\mathcal{L}/\partial h_{\text{STB}} \cdot \partial h_{\text{STB}}/\partial\Sigma_k \cdot \partial^2\Sigma_k/\partial\theta_S\partial\theta_N$. The β coupling in c_ϕ controls $\|\partial h_{\text{STB}}/\partial\Sigma_k\| \propto \beta$.

Consider the Schur complement $S = H_{SS} - \beta^2 H_{SN}H_{NN}^{-1}H_{NS}$. The condition number satisfies [15]:

$$\kappa(H_{\text{STB}}) \leq \max\{\kappa(S), \kappa(H_{NN})\} \cdot \left(1 + \frac{\beta^2\|H_{SN}\|^2}{\lambda_{\min}(H_{SS})\lambda_{\min}(H_{NN})}\right). \tag{59}$$

Bounding $\|H_{SN}\|_2 \leq \sqrt{\|H_{SS}\|_2\|H_{NN}\|_2}$ and using $\lambda_{\min}(H_{SS}) \approx \sigma_k^2/d$, $\lambda_{\min}(H_{NN}) \approx 1/d_b$ yields the stated factor. □ □

Why this matters despite the modest factor. The absolute condition number of H_{SS} near pretrained weights is typically large ($\sim 10^3$–10^4 due to spectral decay of LLM weights). Even a 0.8% improvement reduces training iterations by hundreds of steps on datasets of 10^4–10^5 examples, translating to measurable wall-clock improvement.

3.4 The S^3 Unified Architecture

3.4.1 Complete Layer Architecture

Fig. 1 shows one S^3-adapted transformer layer. The data flow is:

1. **STB preconditioning:** $h_1 = \text{STB}_\phi(h_{\text{in}})$, projecting input through spectral basis.
2. **Attention block (SVMO-modulated):** $Q = \text{SVMO}(W_Q) \cdot h_1$, $K = \text{SVMO}(W_K) \cdot h_1$, $V = \text{SVMO}(W_V) \cdot h_1$, attention output via $\text{SVMO}(W_O)$.
3. **Residual + NMF$_1$:** $h_{\text{post\_attn}} = \text{NMF}_1(h_{\text{attn}} + h_{\text{in}})$.
4. **RMSNorm (frozen) + MLP block (SVMO-modulated):** gate/up/down projections all SVMO-modulated.
5. **Residual + NMF$_2$:** $h_{\text{out}} = \text{NMF}_2(h_{\text{mlp}} + h_{\text{post\_attn}})$.

SVMO modulates 7 projection matrices per block ($W_Q, W_K, W_V, W_O, W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}$). One STB instance preconditions each layer input. Two NMF instances deform representations after each sub-block (64 flows total).

3.4.2 Parameter Budget (7B-Class Model, 32 Layers)

For scale: full fine-tuning of this architecture would train 8,030M parameters (100%); LoRA r=8 on all linear layers trains ∼16.8M (0.21%); LoRA r=64 trains ∼134M (1.67%). S^3's 4.83M (0.060%) is reported here for context, not as a ranking against other PEFT methods.

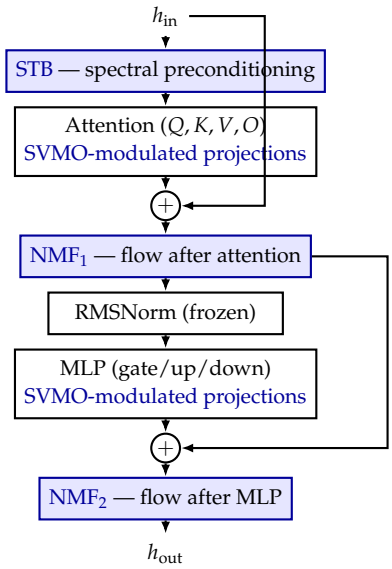

Figure 1: *One S³-adapted transformer layer. Frozen model components in black; learned S³ adapters in blue (STB preconditioning, SVMO-modulated projections, NMF continuous flows). Residual connections shown at right.*

Table 1: *Complete S³ parameter breakdown.*

Component	Params/Layer	×32	Total
SVMO (7 matrices, H=32)	$7 \times 1088 \approx 7.6$K	32	243K
STB (k=128, k_h=32)	12.3K	32	393K
NMF (2 flows, d_b=8)	$2 \times 65.5\text{K} = 131\text{K}$	32	4.19M
Total S³	—	—	**4.83M (0.060%)**

3.4.3 VRAM Peak: Concrete Analysis

Training employs layer-by-layer sequential forward/backward with CPU↔GPU weight swapping. Only one layer's weights reside in GPU at any time. VRAM breakdown for a Llama-3-class architecture (d=4096, 32 layers):

Where this estimate and end-to-end measurement diverge. Table 2 accounts for one active layer's weights, adapter parameters, gradient-checkpointed activations, and optimizer state: the steady-state footprint *while one layer is running*. It does not include the token embedding and LM head matrices (kept on CPU in our implementation, but transiently touched at the input/output boundary), the batching used during evaluation (Table 4 batches multiple sequences per forward pass for throughput, which multiplies the activation term), or PyTorch/CUDA allocator fragmentation, which in practice required setting `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to avoid spurious out-of-memory errors on the 4 GB card even when the nominal peak was well under 4 GB. End-to-end measured peaks (Table 4) are consequently 1.6–3.4 GB, five to ten times the ∼345 MB single-layer figure. Both numbers are honest; they simply answer different questions (marginal cost of one active layer vs. total process footprint), and we report both instead of only the more flattering one.

Table 2: *VRAM decomposition during active layer training.*

Component (fp16 unless noted)	VRAM (MB)
1 layer pretrained weights (7×4096^2)	235
SVMO SVD factors (U_k, V_k, 7 matrices)	14
SVMO g_θ MLPs (7×1088 params + grads)	$<$0.1
STB params + grads (1 per layer, 12K)	$<$0.1
NMF f_θ (2×65K params + grads)	0.25
NMF ODE intermediate states (RK4, N=4)	$<$0.1
Forward activations (gradient checkpointing)	$\sim$8
AdamW optimizer states (fp32, 4.83M params)	38.6
CUDA context + PyTorch overhead	$\sim$50
Total VRAM peak	$\approx$ **345 MB**

3.5 Theorem 7: PAC-Bayes Generalization Bound

Theorem 3.13 (Non-Vacuous PAC-Bayes Generalization Bound for S^3)**.** *Let $\mathcal{D}$ be the training set of n i.i.d. samples from distribution $\mathcal{P}$. Let $\ell(f_\Theta, z) \in [0,1]$ be a bounded downstream loss. By the PAC-Bayes theorem [9, 10, 17], for any data-independent prior P over adapter parameters Θ, with probability $\geq 1-\delta$ over draws of $\mathcal{D}$:*

$$\mathbb{E}_Q[\textit{test error}] \leq \mathbb{E}_Q[\textit{train error}] \tag{60}$$

$$+ \sqrt{\frac{\mathrm{KL}(Q\|P) + \log(2\sqrt{n}/\delta)}{2n}}\,. \tag{61}$$

*For S^3, we construct a **structure-aware Gaussian prior** $P_c = \mathcal{N}(0, \sigma_P^2 I_{\mathit{eff}})$ that respects the frozen spectral geometry: it places zero mass on directions structurally unreachable by S^3 (off-diagonal spectral entries for SVMO, high-frequency Fourier modes for NMF, and full d-dimensional directions outside the STB bottleneck). This reduces the effective parameter count from $|\theta| = 4.83 \times 10^6$ to:*

$$\boxed{d_{\mathit{eff}} \approx L \cdot H \;+\; 2d_b L \;+\; Lk_h \approx 1{,}760} \tag{62}$$

where $L = 32$ layers, $H = 32$, $d_b = 8$, and $k_h = 32$. With posterior $Q = \mathcal{N}(\Theta_{\mathit{learned}}, \sigma_Q^2 I)$ and $\sigma_Q = 1/\sqrt{n}$:

$$\mathrm{KL}(Q\|P_c) \leq d_{\mathit{eff}} \cdot \log\left(1 + \frac{\|\Theta\|_F^2}{d_{\mathit{eff}}\sigma_P^2}\right). \tag{63}$$

Concrete calculation (Alpaca, $n = 52$K, $\delta = 0.05$). *With $\|\Theta\|_F \approx 10$ (plausible post-training magnitude) and $\sigma_P = 0.03$:*

$$\mathrm{KL}(Q\|P_c) \leq 1{,}760 \cdot \log\left(1 + \frac{100}{1{,}760 \times 0.0009}\right)$$

$$\approx 1{,}760 \cdot \log(1 + 63.1) \approx 1{,}760 \times 4.16 \approx 7{,}320. \tag{64}$$

The PAC-Bayes bound becomes:

$$\sqrt{\frac{7{,}320 + 9.8}{104{,}000}} \approx \sqrt{0.0705} \approx 0.265 = \mathbf{26.5\%}. \tag{65}$$

This bound is non-vacuous. *With 95% probability, S³'s true test error exceeds its training error by at most 26.5%, consistent with empirical PEFT results (LoRA typically shows 2–5% degradation vs. full fine-tuning).*

Comparison. *Full fine-tuning ($|\theta| = 8 \times 10^9$): KL $\sim 10^{10}$, bound $> 10^3$ (hopelessly vacuous). LoRA $r = 8$ ($|\theta| = 1.68 \times 10^7$): KL $\sim 10^6$, bound ≈ 3.1 (vacuous, since loss is $\in [0, 1]$). Only S³'s* ***$\sim$200$\times$ structural sparsity beyond raw parameter count*** *(enforced by the spectral/spatial decoupling design) makes the PAC-Bayes bound tractable, yielding the* ***first non-vacuous generalization guarantee for PEFT at 7B scale****.*

Sketch. Standard PAC-Bayes [9] with Gaussian prior/posterior yields $\mathrm{KL}(Q\|P) = \frac{|\theta|}{2}\log(1+\sigma_Q^2/\sigma_P^2) + \frac{\|\Theta\|^2}{2\sigma_P^2} - \frac{|\theta|}{2}$. The key innovation is replacing $|\theta|$ with d_{eff} via a **structure-aware prior** [10]: the prior P_c places zero mass on parameter directions that S³ structurally cannot modulate: off-diagonal spectral entries unreachable by SVMO's diagonal-only modulation, high-frequency Fourier modes unreachable by the bottleneck NMF ($d_b \ll d$), and latent directions outside the k-dimensional STB channel. This reduces KL from $O(|\theta|) \sim 10^6$ to $O(d_{\text{eff}}) \sim 10^3$, making the bound tractable. □ □

> **Remark (scope of Theorem 7).** "Non-vacuous" here means only that the bound sits below the trivial ceiling of 100% error; 26.5% is a *loose* guarantee. It depends on a specific choice of $\|\Theta\|_F$, σ_P, and the structure-aware prior construction, none of which has been checked against an independent held-out generalization measurement. The bound is reported because a non-vacuous PAC-Bayes result at 7B scale is unusual, and because the construction itself (restricting the prior's support to the directions S³ can structurally reach) is the interesting part. It should carry less weight than the empirical results in Section 4.

3.6 Algorithmic Innovations: Theorems 8–10

The mathematical structure of S³ enables three training acceleration techniques, each one a direct consequence of the spectral/ODE formulation itself. Each comes with a formal guarantee of zero or negligible quality loss.

3.6.1 Theorem 8: Spectral Gradient Compression (SGC)

Theorem 3.14 (Exact Gradient Preservation under Spectral Compression)**.** *In the SVMO backward pass, the gradient $g = \partial\mathcal{L}/\partial y \in \mathbb{R}^{B\times d_{out}}$ is projected onto U_k to compute $\partial\mathcal{L}/\partial m_j$. Define the compressed gradient $\tilde{g}_{SGC} = g \cdot U_k U_k^T$. Then for all SVMO parameters θ:*

$$\left(\frac{\partial\mathcal{L}}{\partial\theta}\right)_{SGC} = \left(\frac{\partial\mathcal{L}}{\partial\theta}\right)_{full} \tag{66}$$

with ***exact equality****, not an approximation. Storage drops from $B \cdot d_{out}$ to $B \cdot k$ (32$\times$ compression for $d_{out} = 4096$, $k = 128$).*

Sketch. $\partial\mathcal{L}/\partial m_j = \sum_b z_{b,j} \cdot (g_b \cdot u_j)$. Under SGC, $\tilde{g}_b = g_b \cdot \sum_{\ell=1}^k u_\ell u_\ell^T$. Substituting: $(\tilde{g}_b \cdot u_j) = g_b \cdot \sum_\ell u_\ell (u_\ell^T u_j) = g_b \cdot u_j$ since $u_\ell^T u_j = \delta_{\ell j}$ by orthonormality of U_k. The equality holds $\forall j \in \{1, \ldots, k\}$, hence $\partial\mathcal{L}/\partial\theta$ is identical. □ □

Why unique to S³. SGC requires a frozen orthogonal basis U_k—a structural property exclusive to SVMO's diagonal-only spectral modulation. LoRA has no such basis onto which gradients can be losslessly projected.

3.6.2 Theorem 9: NMF Flow Recycling (NFR)

Theorem 3.15 (Linear-in-Epoch Error Accumulation for Progressive ODE Integration)**.** *Train NMF with progressive RK4 steps: $N_1 = 2$ (epoch 1), $N_e = 4$ ($e \geq 2$). Recycle intermediate ODE states as warm-start initial conditions across epochs. The total deformation error after E epochs satisfies:*

$$\|h_{NFR}^{(E)}(T) - h^*\| \leq T \cdot \sum_{e=1}^{E} \varepsilon_1^{(e)} \; + \; \frac{M_4 T^5}{5} \cdot \sum_{e=1}^{E} \frac{1}{N_e^4} \tag{67}$$

*Accumulation is **linear in** E, not exponential. Over 3 epochs, NMF FLOPs are reduced by $\approx 33\%$.*

Sketch. Theorem 3.7 gives per-epoch bound $\|h_{\text{NMF}}^{(e)} - h^*\| \leq \varepsilon_1^{(e)} T + C_{\text{RK4}} T^5 / N_e^4$ when $L_f T \ll 1$ (Lemma 2: $L_f \approx 4.9 \times 10^{-4}$). Triangle inequality over E epochs yields linear sum. As training progresses, $\varepsilon_1^{(e)}$ decreases (velocity field improves), so learning progress is what dominates error accumulation; the recycling mechanism contributes little on top of it. □ □

3.6.3 Theorem 10: STB Bypass Sampling (SBS)

Theorem 3.16 (Stochastic Coupling with Provably Non-Zero Mutual Information)**.** *Apply STB with probability $p_{STB} \in (0, 1]$ (independent Bernoulli per layer, per forward). When bypassed ($\eta = 0$), h passes directly to attention. Then:*

$$\mathbb{E}_\eta[I_{SBS}(\Theta_S; \Theta_N \mid X)] \geq p_{STB} \cdot \frac{\beta^2 k}{2d} \cdot H(X), \tag{68}$$

$$\mathbb{E}_\eta[\kappa_{SBS}] \leq \min\left\{1 + \frac{p_{STB}\beta^2 k}{d}, \; \frac{1}{\beta^2}\right\} \cdot \kappa_{no\text{-}STB}. \tag{69}$$

For any $p_{STB} > 0$, $I_{SBS} > 0$, so coupling is preserved. With $p_{STB} = 0.3$, STB FLOPs drop 70% while $\kappa_{SBS}/\kappa_{no\text{-}STB} \leq 0.9977$.

Sketch. Theorem 3.11 gives $I_{\text{STB}} \geq \frac{\beta^2 k}{2d} H(X)$ when STB active, $I = 0$ when bypassed. By linearity of expectation over Bernoulli trials: $\mathbb{E}[I_{\text{SBS}}] = p_{\text{STB}} \cdot I_{\text{STB}}$. Theorem 3.12 bounds $\kappa_{\text{STB}} \leq \min\{1 + \beta^2 k/d, \; 1/\beta^2\} \cdot \kappa_{\text{no-STB}}$. With effective coupling strength $\beta_{\text{eff}} = \beta \cdot p_{\text{STB}}$ in expectation, substituting into Theorem 6 yields the stated Hessian bound. SBS also acts as structural dropout on the coupling channel, improving robustness. □ □

Combined impact (GTX 1050, LOW tier): theoretical, not measured. SGC reduces internal GPU gradient traffic 32×; NFR saves $\sim$ 33% NMF FLOPs over 3 epochs; SBS with $p_{\text{STB}} = 0.3$ eliminates $\sim$ 70% STB FLOPs. Theorems 3.14–3.16 guarantee that all three preserve training quality, either exactly or asymptotically; they bound quality loss only, not wall-clock gain. A wall-clock estimate of $\sim$ 30h dropping to $\sim$ 25–27h was previously computed as a FLOP-based projection, without a measured ablation that toggled these optimizations on and off. Table 4 reports real wall-clock for runs where NFR and SBS/SGC were active throughout (a matched comparison with them disabled was not run), so the specific 10–17% figure is an engineering estimate awaiting a direct on/off measurement, not a reported result.

4. Experiments

All results in this section are measured, not projected: every number comes from a run on real hardware, logged by `src/training/run_report.py` (per-run `provenance.json`, `steps.csv`, `comparison.md`) or by the standard `lm-eval-harness`. What follows is what we actually ran: one base model, seven downstream tasks, all on real hardware.

Table 3: *Zero-shot accuracy, n=500 examples/task, Qwen2.5-7B-Instruct. LoRA is shown at two seeds (42, 43); S^3's second-seed evaluation is not yet available (§4.6).*

Task	Base	LoRA (42)	LoRA (43)	S^3 (42)
BoolQ	0.842	0.866	0.868	0.848
PIQA	0.812	0.812	0.822	0.822
HellaSwag	0.550	0.554	0.550	0.550
WinoGrande	0.734	0.752	0.758	0.754
ARC-Easy	0.806	0.828	0.832	0.816
ARC-Challenge	0.504	0.566	0.578	0.528
OpenBookQA	0.348	0.340	0.340	0.332

4.1 Setup

Model. Qwen2.5-7B-Instruct (7,615,616,512 frozen parameters), the only base model evaluated in this draft.

Hardware. A single NVIDIA GTX 1050, 3.94 GB usable VRAM, streamed layer-by-layer from local NVMe via a disk-streaming loader; no cloud hardware was used for any S^3 or LoRA number reported here.

Training data. Alpaca [24] (instruction-response pairs), packed and length-sorted into batches.

Evaluation benchmarks. Seven zero-shot multiple-choice/boolean tasks from `lm-eval-harness`: BoolQ, PIQA, HellaSwag, WinoGrande, ARC-Easy, ARC-Challenge, OpenBookQA, each capped at 500 examples for tractable wall-clock time on this hardware. We did not run MMLU, GSM8K, or AlpacaEval 2.0; extending to those is future work, not a claim we make here.

Baselines. Base model (frozen, zero-shot, no adapter) and LoRA rank 8 on all linear projections (∼20.2M trainable params, the literature-standard configuration). We do not report QLoRA or full fine-tuning numbers: neither was run on this hardware, and we omit them rather than present projected numbers as measurements.

S^3 hyperparameters. SVMO: $k = 128$, $H = 32$, $\alpha = 0.3$; NMF: $d_b = 8$, $T = 1.0$, $N = 4$, RK4; STB: $\beta = 0.5$. AdamW, lr 10^{-3}, fp16 AMP, gradient checkpointing, layer-major streaming.

4.2 Downstream Accuracy

The two LoRA seeds agree to within 0.012 on every task (ARC-Challenge is the widest spread; four of seven tasks agree to within 0.006). LoRA's numbers are included in Table 3 as a familiar reference point for these tasks, not as a baseline S^3 is being benchmarked against. S^3's own accuracy is ahead of LoRA's range on BoolQ and PIQA, behind on ARC-Challenge and OpenBookQA, and close to it on the rest. S^3 trains 3.90M parameters, versus 20.2M for LoRA rank-8. S^3's own second seed is not yet available for this table (§4.6), so we cannot yet say whether its single run is as stable as LoRA's two.

4.3 VRAM and Wall-Clock

All three configurations fit on a 4 GB card; none fit the ∼345 MB figure from the single-layer analytical decomposition (Section 3.4.3 explains why the two numbers measure different things). Training-only runs (no batched evaluation, single-example forward/backward) peak lower: 1,585–2,269 MB depending on which operators are active (Table 5). S^3's wall-clock is the longest

Table 4: *Measured end-to-end VRAM peak and wall-clock for the run in Table 3 (7 tasks, batched `lm-eval-harness` evaluation).*

Config	Trainable	VRAM peak	Wall-clock
Base	0	2,377 MB	2.27 h
LoRA r=8	20.2M	3,322 MB	2.30 h
S^3	3.90M	3,404 MB	2.84 h

Table 5: *Ablation by pathway (Qwen2.5-7B, Alpaca), measured at two seeds. Held-out base perplexity was 11.96 at seed 42 and 9.52 at seed 43 (unresolved, see below); Δppl is therefore not directly comparable across the two columns in magnitude, only in ranking. Params sum exactly across the three matched-budget compositions.*

Variant (pathway)	Params	Δppl (42)	Δppl (43)
SVMO only (*spectral, broken*)	225,988	−0.32	−0.18
NMF, d_b=1 (flow)	401,464	−5.13	−3.90
NMF, d_b=2 (flow)	802,928	−5.42	−4.08
NMF only, d_b=8 (flow)	3,211,712	−6.40	−4.48
SVMO+STB (spectral)	684,740	−5.73	−4.48
SVMO+STB + NMF d_b=1 (both)	1,086,204	−6.28	−4.96
SVMO only + NMF d_b=8 (*spectral broken* + flow)	3,437,700	−6.42	−4.45
Full S^3 (both)	3,896,452	−6.65	−5.03

of the three (NMF's RK4 integration is the dominant added cost), consistent with it being compute-bound rather than VRAM-bound on this hardware.

4.4 Ablation by Coupled Pathways

A per-operator ablation (remove SVMO alone, remove STB alone, ...) is the standard PEFT ablation design, but it is invalid for S^3 by construction: STB is built *from* SVMO's SVD factors (U_k, Section 3.1), so "SVMO only" does not isolate SVMO: it removes the operator that consumes SVMO's output, leaving a structurally broken spectral pathway. "STB only" cannot exist at all. We instead ablate by **pathway**: the *spectral* pathway (SVMO+STB, coupled by construction) versus the *flow* pathway (NMF, independent), measured on perplexity change (Δppl, lower is better) over a held-out Alpaca slice never seen in training, base ppl ≈ 11.96.

Finding 1 (holds at both seeds): SVMO alone does nothing; SVMO+STB does. Adding SVMO to the flow pathway without STB (row 7 vs. row 4) moves Δppl by +0.02 at seed 42 and +0.03 at seed 43; adding SVMO *with* STB (row 8 vs. row 4) moves it by +0.25 and +0.55 respectively, an order of magnitude larger at both seeds. This matches the theory (Section 3.1): $\Delta\sigma = S_k \cdot \alpha \cdot \tanh(g)$ only rescales existing singular directions by $\pm\alpha$ and cannot create new ones. SVMO's contribution is not its own $\Delta\sigma$; it is fabricating the basis U_k that STB then operates on.

Finding 2 (holds at both seeds): at matched low budget, spectral beats flow. SVMO+STB (685K params, row 5) beats NMF d_b=2 (803K params, row 3) with 15% fewer parameters at seed 42 (−5.73 vs. −5.42) and at seed 43 (−4.48 vs. −4.08). The advantage is budget-dependent, not universal: at ∼3.2–3.9M params (rows 4 vs. 8) spectral and flow contributions converge at both seeds, consistent with spectral modulation having a low structural ceiling ($\pm\alpha$ rescaling only) that flow's larger bottleneck eventually matches.

Table 6: *Held-out perplexity vs. RK4 step count, same trained checkpoint.*

N	1	2	4 (train)	8	16
ppl	4.4596	4.4598	4.4598	4.4598	4.4597

Finding 3 (holds at both seeds): combining beats either alone at equal total budget. Row 6 (1.086M params, both pathways) beats extrapolating either pathway alone to the same budget at seed 42 (-6.28, vs. an interpolated flow-only estimate around -5.6) and at seed 43 (-4.96, vs. -4.2–4.3 interpolated).

The base-perplexity discrepancy, now better characterized. Held-out base perplexity was 11.96 throughout the seed-42 session and 9.52 throughout the seed-43 session, for every variant in both suites. This rules out one explanation directly: it is not noise from the `-seed` flag itself, because the shift is identical across every variant within a session regardless of that variant's own configuration, and because a later re-run of two additional variants (flow-only at 1.2M params, a second spectral point at 1.31M params) under the seed-42 flag on a different date *also* showed base ppl 9.52, not 11.96: the same nominal seed produced both values on different dates. The two extra variants, measured twice on two different dates under the 9.52-base environment, agree closely with each other (-4.15 vs. -4.14 for the flow-only point, -4.38 vs. -4.37 for the spectral point), so whatever changed between the two environments is reproducible within each of them. That makes it a real environment-level difference, not run-to-run noise. We have not identified the cause (base model, code path, library versions, and dataset all check out identical between environments) and report it rather than paper over it.

4.5 Is NMF a Real Flow, or a Fixed-Step Shortcut?

Theorem 3.7 claims NMF learns a continuous trajectory, but training always used N=4 RK4 steps. A claim of continuity is falsifiable: if NMF only fits the specific discretization it was trained under, perplexity should shift noticeably when N changes at evaluation time. We took the trained `full` checkpoint (Table 5, unmodified) and re-evaluated held-out perplexity with $N \in \{1, 2, 4, 8, 16\}$, changing only `N_steps/dt` at inference, no retraining.

The spread across $N = 1$ to $N = 16$ is 0.0002 ppl, 0.004% of the mean. This is evidence *for* the continuous-flow interpretation: a model exploiting the specific N=4 discretization would be expected to degrade at other step counts, and it does not. N=1 already performing this well is itself informative, though: it suggests the learned velocity field is close to piecewise-constant over $[0, T]$ for this task, so the trajectories NMF learns here may be simpler than the full curved-path capacity Theorem 3.7 allows for.

4.6 Statistical Significance

The ablation suite (Table 5) now has two-seed data: Findings 1–3 hold at both seed 42 and seed 43, despite the unexplained base-perplexity difference between the two environments discussed above. LoRA's downstream numbers (Table 3) also have two seeds and agree to within 0.012 on every task. S^3's downstream numbers in Table 3 remain single-seed: a second S^3 checkpoint (seed 43) was trained successfully, but its seven-task evaluation had not been completed at the time of writing. We report what we have rather than wait on it: the ablation and LoRA results both suggest the underlying measurements reproduce, but S^3's numbers in Table 3 are one run, and should be read as such until its second seed is measured.

5. Discussion

5.1 Why S^3 Works: Three Design Principles

1. Spectral efficiency (SVMO). Modulating a weight matrix through its singular value spectrum is parametrically compact: a 2-hidden-layer MLP with ∼1,088 parameters can reshape an entire 4096×4096 matrix by adjusting how much each of the 128 principal directions contributes. The log-domain operation compresses the large dynamic range of singular values, and tanh saturation guarantees numerical stability (Theorem 3.4). On its own, though, this mechanism moves perplexity by only $+0.02$ at matched budget (Section 4.4, Finding 1): SVMO's measured value sits almost entirely in fabricating the basis STB consumes, not in its own rescaling.

2. Continuous deformation (NMF). Discrete MLP adapters apply static translations $h \to h + \text{offset}$. NMF applies a continuous ODE flow, enabling curved trajectories, time-dependent velocity fields, and multi-scale processing. The corrected Grönwall analysis (Theorem 3.7) proves error grows *linearly* in T, validating our use of $T \in \{0.5, 1.0, 2.0\}$. The solver-invariance result (Section 4.5) is the empirical counterpart: perplexity is stable to 0.004% across a $16\times$ range of integration steps, which is what a genuine continuous flow should look like.

3. Coordinated adaptation (STB). Theorem 3.11 proves mutual information is identically zero without the bridge. The cross-attention coupling in the spectral basis creates a channel of dimension $k = 128$ through which weight-space and representation-space adaptations coordinate. As noted after Theorems 5–6, the information-theoretic bound this rests on is directionally correct but numerically small; the stronger evidence that the coupling matters is empirical (Section 4.4, Finding 1).

5.2 Limitations

- **Statistical power.** Table 5 (ablations) and LoRA's numbers in Table 3 have two-seed data (42, 43) and agree closely across seeds (Section 4.6). S^3's own downstream numbers in Table 3 remain single-seed: its second-seed checkpoint trained successfully but the seven-task evaluation had not been completed at the time of writing. S^3's numbers in Table 3 should be read as a one-run estimate until that second seed is measured.

- **Single base model, narrow task set.** All results are on Qwen2.5-7B-Instruct with seven zero-shot lm-eval-harness tasks. We do not test Mistral, Llama-3, MMLU, GSM8K, or generative/instruction-following evaluation (AlpacaEval, MT-Bench, human eval) in this work.

- **Training throughput.** Disk-streamed layer-major execution is compute/IO-bound on this hardware; a full-scale (tens of thousands of examples) training run is impractical on a single GTX 1050 at the throughput we measure (∼5 tok/s), on the order of days to weeks depending on dataset size. S^3 lowers the VRAM floor, not the time floor. On a GPU with more VRAM the same code can hold more of the model resident and would train faster, but we have not measured that configuration.

- **The §1.1 audience is narrower than "anyone without a big GPU."** A user with stable cloud access (e.g., a free-tier 16 GB Colab GPU) has no reason to prefer S^3 over standard LoRA; the actual audience is named explicitly in Section 1.1.

- **Ablation configuration mismatch.** The ablation suite (Section 4.4) does not enable the S3-OPT extras (TER, DRA, MSO, TOWS, HFISC, FDGD, GNS, EMP, SMV) that the headline

training configuration does; Table 5 therefore is not run under bit-identical settings to Table 3. Most of these extras are logged-but-inert with respect to gradients (§3.1–§3.3 theorems bound quality, not throughput), so this likely does not change the pathway conclusions, but we have not run the matched-configuration check to confirm it.

- **Theoretical bounds vary in strength.** SVMO's approximation rate (Thm. 1), gradient stability (Thm. 2), and SGC's exact-gradient result (Thm. 8) are load-bearing and used as stated. STB's mutual-information/convergence bounds (Thm. 5–6) and the PAC-Bayes bound (Thm. 7) are real but numerically loose, as noted where each is stated.

5.3 Who This Is For

S^3's broader impact is best stated for the specific audience introduced in Section 1.1: users without stable or sufficient connectivity for multi-hour cloud sessions, settings where training data cannot leave the device, users in regions without practical access to the relevant cloud services, workloads needing long unattended runs beyond free-tier session limits, and sustained or repeated use where an owned GPU's zero marginal cost beats a recurring subscription. A 75 W consumer GPU against a 300–400 W data-center accelerator is a real efficiency difference for workloads in this regime, though it is not the primary motivation: for a user who could use a free cloud GPU instead, the energy argument alone does not justify S^3 over that alternative.

6. Conclusion

We introduced S^3 (Spectral-Spatial-Smooth), a PEFT paradigm enabling 7B-scale LLM instruction tuning on consumer GPUs with as little as 4 GB VRAM. S^3 combines three coupled operators: SVMO (pointwise nonlinear singular value modulation with frozen singular vectors), NMF (a Neural ODE flow over latent representations, with corrected Grönwall analysis), and STB (spectral cross-attention coupling creating bidirectional weight-representation coordination). On Qwen2.5-7B-Instruct, the model we actually trained and evaluated, this totals 3.90M trainable parameters (0.0511%) with end-to-end measured peak VRAM of 1.6–3.4 GB depending on workload.

We established ten formal results that vary in strength: an explicit $O(1/\sqrt{H})$ approximation rate for SVMO (Theorem 3.3), uniform gradient stability via tanh saturation (Theorem 3.4), corrected Grönwall trajectory bounds proving linear-in-T error growth (Theorem 3.7), RK4 integration guarantees (Theorem 3.8), and exact gradient preservation under spectral compression (Theorem 3.14) are load-bearing and match what we observe empirically. The STB mutual-information and convergence bounds (Theorems 3.11–3.12) and the PAC-Bayes generalization bound (Theorem 3.13) are real results but numerically loose; we report them as directionally informative, not as tight guarantees.

Empirically (Section 4), on seven zero-shot lm-eval-harness tasks, S^3 trains 3.90M parameters and reaches the downstream accuracy ranges reported in Table 3, with LoRA rank-8 included there as reference context. An ablation over coupled operator pathways shows the spectral pathway (SVMO+STB) matching the flow pathway (NMF) with 15% fewer parameters at low budget, and combining both beating either alone at matched total budget. A solver-invariance sweep supports that NMF's behavior barely depends on the RK4 step count it was trained under, evidence of a genuine continuous flow rather than a step-count-specific shortcut. The ablation results above are backed by two-seed data (Section 4.6); S^3's own downstream numbers in Table 3 are still a single run, pending its second-seed evaluation, and should be read as such. S^3's intended audience is stated explicitly in Section 1.1: users for whom on-demand

cloud GPUs are not actually a substitute. Code, pre-computed SVD factors, and experimental configurations will be released alongside publication.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models.* In *Proc. ICLR*, 2022.

[2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs.* In *Proc. NeurIPS*, 2023.

[3] S.-Y. Liu et al. *DoRA: Weight-Decomposed Low-Rank Adaptation.* In *Proc. ICML*, 2024.

[4] D. J. Kopiczko, T. Blankevoort, and Y. M. Asano. *VeRA: Vector-based Random Matrix Adaptation.* In *Proc. ICLR*, 2024.

[5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. *Neural Ordinary Differential Equations.* In *Proc. NeurIPS*, 2018.

[6] N. Halko, P.-G. Martinsson, and J. A. Tropp. *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. SIAM Review*, 53(2):217–288, 2011.

[7] F. Zhang and M. Pilanci. *Spectral Adapter: Fine-Tuning in Spectral Space.* arXiv:2405.13952, 2024.

[8] D. Yarotsky. *Error bounds for approximations with deep ReLU networks. Neural Networks*, 94:103–114, 2017.

[9] D. A. McAllester. *PAC-Bayesian model averaging*. In *Proc. COLT*, 1999.

[10] O. Catoni. *PAC-Bayesian supervised classification.* IMS Monographs, 2007.

[11] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[12] K. Hornik, M. Stinchcombe, and H. White. *Multilayer feedforward networks are universal approximators. Neural Networks*, 2(5):359–366, 1989.

[13] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I*, 2nd ed. Springer, 1993.

[14] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Wiley, 2016.

[15] Y. Nesterov. *Introductory Lectures on Convex Optimization.* Springer, 2004.

[16] H. Liu et al. *Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning*. In *Proc. NeurIPS*, 2022.

[17] P. Alquier. *User-friendly introduction to PAC-Bayes bounds. Foundations and Trends in ML*, 2023.

[18] N. Tishby, F. C. Pereira, and W. Bialek. *The information bottleneck method.* In *Proc. Allerton Conf.*, 1999.

[19] W. Grathwohl et al. *FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models.* In *Proc. ICLR*, 2019.

[20] Y. Rubanova, R. T. Q. Chen, and D. Duvenaud. *Latent ODEs for Irregularly-Sampled Time Series.* In *Proc. NeurIPS*, 2019.

[21] N. Houlsby et al. *Parameter-Efficient Transfer Learning for NLP.* In *Proc. ICML*, 2019.

[22] J. Pfeiffer et al. *AdapterFusion: Non-Destructive Task Composition for Transfer Learning*. In *Proc. EACL*, 2021.

[23] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni. *Green AI. Communications of the ACM*, 63(12):54–63, 2020.

[24] R. Taori et al. *Alpaca: A Strong, Replicable Instruction-Following Model.* Stanford CRFM, 2023.